

Project 3: Synthetic-to-Real Chessboard Image Generation from FEN

Final Report

Authors: Tal Podgor, Ilay Vanunu, Ran Packler | Course: Intro to Deep Learning, BGU

Repository: https://github.com/TalPodgor/DL_Project3_final

Project website: https://talpodgor.github.io/DL_Project3/

Abstract

This project addresses chessboard image generation from a FEN position (FEN, or Forsyth-Edwards Notation, is the standard one-line text encoding of a chess position that records which piece occupies each of the 64 squares) and a requested camera viewpoint. The required output is a clean synthetic chessboard image and a more realistic image that preserves the same board state. We treat the problem as controlled synthetic-to-real image translation. Early unpaired translation with CUT/CycleGAN-style training produced realistic texture but did not reliably preserve exact board structure. The final system therefore uses a paired, pix2pixHD-style conditional generator, augmented with chess-specific semantic and geometric conditioning from Blender: an RGB render, a FEN semantic layout, a rendered silhouette, a depth-like map, and piece-edge cues. The selected checkpoint, chess_v5_bright_silABC, is trained on a camera-aligned Blender dataset with per-game oblique elevations of 38-44 degrees. The model is trained on games 4-7 and evaluated on the fully held-out game 2, so the test positions come from a game never seen during training. On these 140 held-out game-2 boards it achieves 0.922 square accuracy, 0.992 occupancy accuracy, 0.804 occupied-piece type accuracy, and 0.757 whole-board occupancy exactness under an independent square classifier. Qualitatively, the model gives the best practical trade-off between realistic wooden-board appearance and board-state preservation, although piece topology artifacts remain a central limitation.

1. Introduction

The task in Project 3 is to implement `generate_chessboard_image(fen, viewpoint)`. Given a FEN string and a viewpoint string, the function must save `synthetic.png`, `realistic.png`, and `side_by_side.png` under `./results/`. The generated realistic image should look like a photograph of a wooden chessboard, the board style of the real photographs in our dataset, while preserving the exact piece placement and the requested white-side or black-side view.

This task is harder than ordinary style transfer. A visually plausible chessboard is not sufficient: a pawn cannot appear on an empty square, a rook cannot become a bishop, and the camera orientation must remain consistent with the input viewpoint. At the same time, the real training photographs are low resolution, strongly rectified to the board plane, and contain small, low-contrast pieces. Tall pieces lean across square boundaries after rectification, so the board grid can be aligned while piece heads and bodies are not perfectly aligned.

Main contributions

- A FEN-controlled Blender rendering pipeline that creates synthetic chessboards and auxiliary control maps.
- A paired pix2pixHD-style translation model that conditions on RGB, semantic, depth, silhouette, and edge information.
- A camera-aligned V5 dataset that replaces near-top-down synthetic supervision with oblique Blender renders closer to the real photos.
- A set of quantitative and visual ablations showing why unpaired translation, silhouette-only conditioning, geometry locking, and naive parallax stretching were rejected.

Figure 1 gives a high-level overview of the whole approach in plain terms. From a FEN and a viewpoint we render a synthetic 3-D board three aligned ways (its appearance, the exact identity of the piece on each square, and a depth map of its 3-D shape), pack these into a single rich per-pixel input, and let a trained

translator repaint it into a realistic photograph while keeping the position unchanged. The remaining sections formalize each stage.

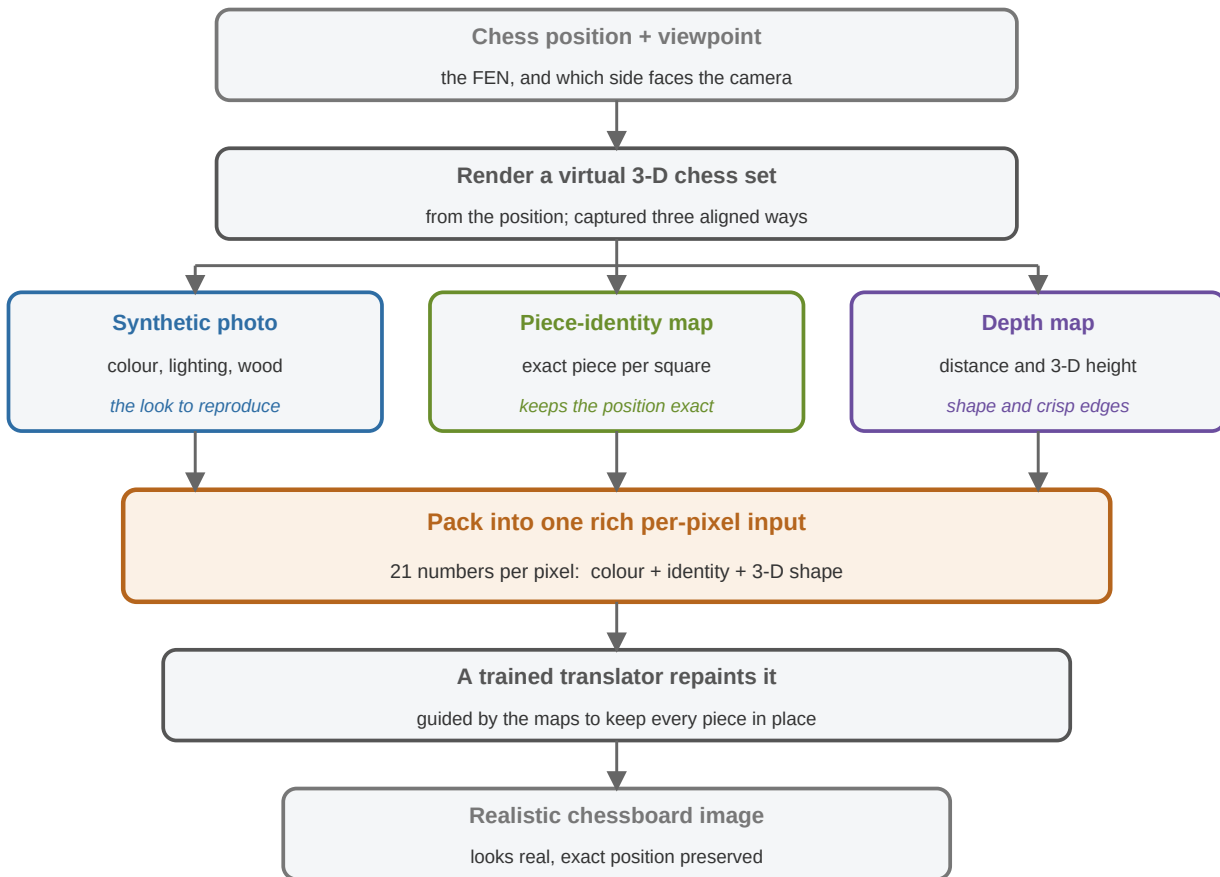


Figure 1. High-level overview of the method. A FEN and viewpoint drive a synthetic 3-D render captured three aligned ways (appearance, per-square piece identity, and depth); these are packed into one rich per-pixel input (21 values per pixel) that a trained, paired translator turns into a realistic image with the exact position preserved. Figure 2 shows the same pipeline in technical detail.

2. Related Work

Image-to-image translation with conditional GANs comes in two families that differ mainly in whether they require aligned input-target pairs. The paired family learns a direct mapping from an input image to its matching target. Pix2pix [1] introduced this setup: a generator trained with an adversarial loss from a conditional PatchGAN discriminator together with an L1 reconstruction term that ties the output to the ground-truth target pixel-for-pixel. Pix2pixHD [2] scales the same idea to higher resolution and stabilizes training with two additions we also rely on: a multi-scale discriminator (several discriminators operating at different image scales) and a feature-matching loss that aligns the discriminator's intermediate activations between the real and generated images. In practice pix2pixHD produces sharper, more stable results and is better suited to detailed targets.

The unpaired family removes the need for aligned pairs and instead learns to translate between two unaligned image collections. CycleGAN [3] trains two generators (a forward and a backward mapping) tied by a cycle-consistency loss: translating an image to the other domain and back should recover the original, which enables style transfer without paired data. CUT [4] simplifies this to a single generator and replaces cycle consistency with a patchwise contrastive loss that keeps corresponding patches of the input and output close in a learned feature space, making it faster and one-directional. The shared limitation for our task is that neither receives a direct, localized penalty for moving a piece or inventing content: they optimize domain-level style, so the object layout can drift.

Our final method is closest to pix2pixHD: paired supervision with a conditional generator, a multi-scale discriminator, and feature matching. The key difference from prior work is the input. Rather than translating from RGB alone, we condition the generator on chess-specific control maps derived from the FEN (a semantic layout, a rendered piece silhouette, a depth-like map, and piece edges), which give the model an

explicit, position-accurate description of what must appear where. We evaluated CUT and CycleGAN as early baselines but did not select them, because the task is not only to make a synthetic board look real, but to preserve the exact position, which unpaired training does not guarantee.

3. Data and Preprocessing

The final training set is `datasets/chess_v5_oblique_aligned_bright`. It contains paired images where the left half is the Blender-rendered source and the right half is the real target. Each sample also has a semantic silhouette image and a depth-like image. The split contains 736 training pairs from games 4, 5, 6, and 7, and 140 held-out test pairs from game 2. The held-out game split is important because it tests whether the model generalizes to unseen positions and viewpoint instances rather than memorizing one board sequence.

A central empirical finding was that camera geometry matters. The older V5 oblique dataset used a global elevation of approximately 54.9 degrees, while the real board photographs were more oblique. The final aligned-bright dataset uses per-game elevations of 38, 40, 42, and 44 degrees with a mean of 40.7 degrees. This reduces the mismatch between the synthetic piece projection and the real rectified photographs.

Split	Games	Pairs	Notes
Train	4, 5, 6, 7	736	Paired Blender source, real target, semantic silhouette, depth
Test	2	140	Held-out game used for quantitative and qualitative evaluation

Dataset	Camera elevation	Resulting role
<code>chess_v5_oblique</code>	54.9 deg global	Rejected as too top-down for real piece geometry
<code>chess_v5_oblique_aligned_bright</code>	38-44 deg per game	Final training data for <code>bright_silABC</code>

4. Method

The final model is called `paired_geom_hd` in the codebase. It is a paired conditional image-to-image model in the `pix2pixHD` family, not a CycleGAN. Like all GAN-based translators, it is built from two networks that are trained together: a generator, which produces images, and a discriminator, which judges them. Only the generator is kept for inference; the discriminator exists only to train it.

Figure 2 shows the full inference pipeline used by `generate_chessboard_image(fen, viewpoint)`: starting from only a FEN and a viewpoint, Blender renders the RGB, semantic, and depth images, these are turned into the 21-channel conditioning, the generator `G` produces the realistic image, and the synthetic and side-by-side outputs are saved. The rest of this section explains each stage of this pipeline in detail.

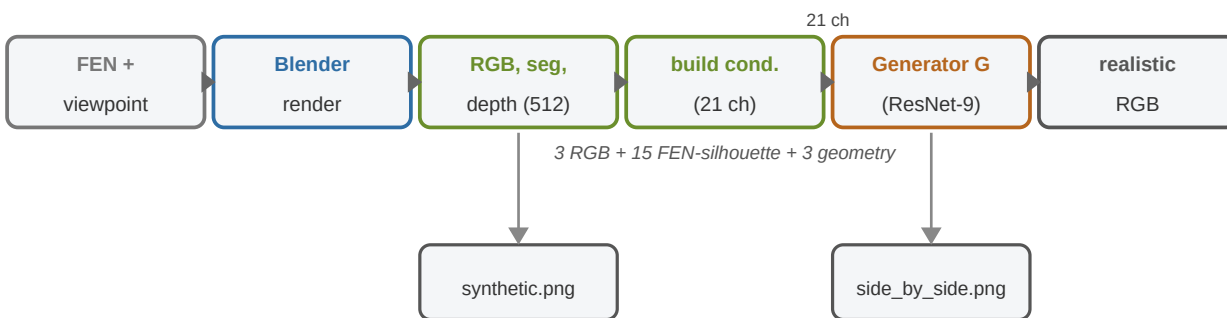


Figure 2. Inference pipeline. From the FEN and viewpoint, Blender renders an RGB image plus a semantic-silhouette and a depth map; these are turned into the 21-channel conditioning tensor. The generator `G` outputs the realistic RGB. The synthetic render and the realistic output are also written and tiled into the side-by-side comparison.

Generator and discriminator

The generator is a ResNet network with 9 residual blocks and instance normalization. This is the same generator architecture that was introduced for neural style transfer and later used by CycleGAN and CUT; we reuse the architecture but train it in a paired setup. It works as an encoder-transform-decoder: an encoder of strided convolutions compresses the 21-channel input into a compact feature map, the 9 residual blocks transform that representation while preserving the spatial layout, and a decoder of

up-sampling convolutions expands it back to a full-resolution three-channel RGB image. Its job is to take the synthetic conditioning and output a realistic-looking board that obeys it.

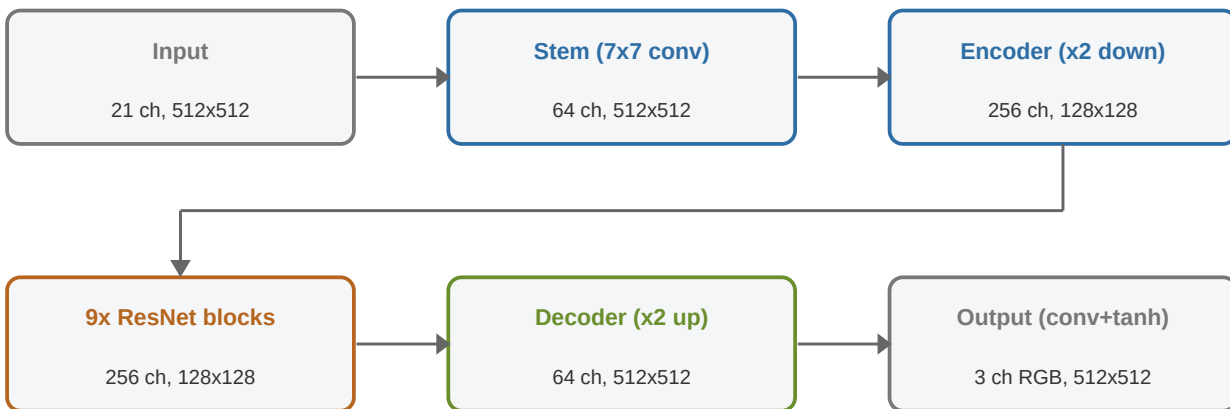


Figure 3. Internal structure of the generator (the "Generator G" block from Figure 2). An encoder (stem plus two downsampling steps) compresses the 21-channel input down to a 128x128, 256-channel representation; nine residual blocks transform it at that same size; a decoder (two upsampling steps plus the output convolution) expands it back to a full-resolution, 3-channel realistic RGB image.

The discriminator is a PatchGAN. Instead of looking at the whole image and emitting a single real-or-fake score, a PatchGAN outputs a grid of scores, each one judging a small local patch of the image. This makes it focus on local realism, the texture of the wood, the grain of the squares and the edges of the pieces, rather than on global composition, which is exactly what matters for a convincing photograph. During training the discriminator is shown both real game photographs and the generator's outputs, and learns to tell them apart.

The two networks are trained against each other, which is why both are needed. The discriminator is optimized to label real photos as real and generated images as fake; the generator is optimized to produce images the discriminator labels as real. This adversarial pressure is what drives realism: a generator trained only with a pixel-reconstruction loss tends to output blurry averages, because a blurry image minimizes the average pixel error, whereas the discriminator rewards sharp, photo-like detail because that is what separates real from fake. As training proceeds the two improve together until the generator's outputs are hard to distinguish from real photographs. At inference the discriminator is discarded and only the generator is run.

Multi-scale and local piece-crop discriminators

We use two extensions of this idea. First, a multi-scale PatchGAN: rather than a single discriminator, we run two PatchGAN discriminators on the image at two different resolutions, the full image and a down-sampled copy. The full-resolution one polices fine detail, while the coarse one polices larger structure that a small patch would miss. This is the pix2pixHD design and gives both sharp texture and globally coherent output. Second, a local piece-crop discriminator, used only during training. The global discriminators see the whole board, where each piece is small; the piece-crop discriminator instead zooms into individual occupied squares (96-pixel crops) and judges each piece on its own, conditioned on the piece type that should be there. This puts additional pressure exactly where the task is hardest, on the pieces, and penalizes a generated blob that looks board-realistic overall but is not a recognizable, correct piece. Like the other discriminators it is discarded at inference.

Input representation

For each position, the model receives a tensor formed by concatenating: (1) the RGB synthetic render, (2) a one-hot semantic map, and (3) three geometry channels. The semantic map uses fen_silhouette mode: full-cell FEN information protects board occupancy, while rendered silhouette pixels provide a more accurate outline of visible piece shapes. The geometry channels are a depth-like render, a piece silhouette mask, and a silhouette boundary channel.

Concretely, every board position is passed through Blender once to produce three aligned 512x512 renders (Figure 4): the RGB image; a semantic-silhouette image in which each of the twelve piece types emits a

distinct flat colour; and a depth-like image whose grayscale encodes distance from the camera. From these, the conditioning is assembled: the semantic image, combined with the FEN layout, becomes the fifteen one-hot channels, while the depth image together with a silhouette mask and its boundary (both derived from the semantic image) form the three geometry channels. All three renders are produced from the FEN and viewpoint alone, with no access to any real photograph, so the exact same inputs are available at training and at inference time; the real photograph is used only as the training target.

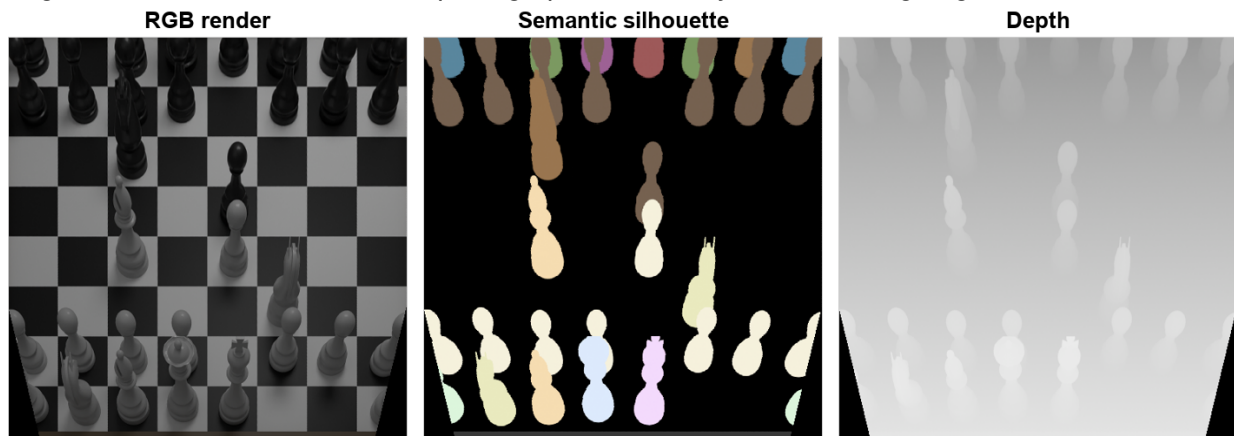


Figure 4. The three aligned Blender renders produced for a single FEN (here `r1bqkbnr/pppp1ppp/2n5/4p3/2B1P3/5N2/PPPP1PPP/RNBQK2R`, white viewpoint). Left: the RGB synthetic render. Middle: the semantic-silhouette render, where each piece type emits a distinct colour (white pieces in warm tones, black pieces in dark browns, queen and king highlighted). Right: the depth render, brighter for surfaces closer to the camera. The same three renders are produced at both training and inference.

Training losses

The generator is not trained with one loss but with a weighted sum of several. Each term compares the generated image to something different (the real photo's pixels, the real photo's deep features, the discriminator's opinion, the piece outline, ...), so each one pulls the generator towards a different notion of "correct". No single comparison captures everything we care about: a loss that only matches raw pixels keeps the position and colour right but, as explained above, tends to blur fine detail; a loss that only asks the discriminator "does this look real" can drift away from the exact target. Combining several gives each its specific job. Table 1a lists the losses computed on the whole image.

Loss	Compares	Plain-English purpose	Weight
Adversarial (GAN)	generator output vs. the multi-scale discriminator's real/fake judgment	Pushes the whole image towards looking like a real photo; not tied to one target pixel, so it rewards general photographic style.	1.0
Feature matching	discriminator's internal activations, real vs. fake	A softer, more stable companion to the adversarial loss; adds structural pressure and helps training converge smoothly.	10.0
VGG perceptual	deep features from a pretrained image-recognition network (VGG), fake vs. real	Rewards correct texture and content even when pixels do not align exactly; more forgiving than comparing raw pixels one-to-one.	3.0
Masked L1 (pixel)	raw pixel colours, fake vs. real, pixel by pixel	Directly anchors colour and position to the real target. The weight is lowered on piece pixels (to 0.7x) because parallax shifts the real piece heads slightly; a full-strength pixel loss there would force blur to average out the mismatch.	7.5 (x0.7 on pieces)
Edge (silhouette boundary)	outline of the rendered piece silhouette vs. the corresponding region in the output	Keeps piece edges crisp, directly fighting the blur the pixel and perceptual losses can introduce. This is component B.	1.0
Contextual (CX)	VGG features again, but matched to the best-fitting nearby feature rather than the exact same pixel position	Tolerates the small parallax misalignment while still rewarding correct local texture. This is component C.	0.5

Table 1b lists the local piece-crop losses. These repeat the same four ideas (adversarial, feature matching, perceptual, and a direct identity check) but computed only on the 96-pixel occupied-square crops described

above, using the piece discriminator rather than the global one. They exist because the global losses average over the whole board, where each piece is a small fraction of the image; the local versions put the same pressure directly on the pieces, which is where legibility problems concentrate.

Loss	Compares	Plain-English purpose	Weight
Piece adversarial	piece crop vs. the piece discriminator's real/fake judgment, conditioned on piece type	Same idea as the adversarial loss above, zoomed into one piece at a time.	1.0
Piece feature matching	piece discriminator's activations, real vs. fake crop	Same idea as feature matching above, but for individual pieces.	10.0
Piece VGG	deep features inside the crop only	Same idea as the VGG perceptual loss above, but for individual pieces.	5.0
Frozen classifier	predicted piece class, from an independently pretrained classifier, vs. the true class from the FEN	Directly rewards "this looks like the correct piece type", not just "looks like some piece".	0.5

In short: the pixel and perceptual losses anchor colour, texture and position; the contextual and edge losses are added specifically to tolerate the parallax mismatch while keeping piece outlines sharp; the adversarial and feature-matching losses push the whole image towards general photographic realism; and the piece-level losses repeat that same pressure zoomed into individual pieces, because that is where the model's hardest remaining problem lives.

Parameter	Final value
Model	paired_geom_hd
Dataset mode	v5_oblique
Semantic source	fen_silhouette
Generator	ResNet 9-blocks, ngf=64, instance norm
Batch size	1
Epochs	40 + 20 decay
lambda_GAN, lambda_feat, lambda_VGG, lambda_L1	1.0, 10.0, 3.0, 7.5
l1_piece_w	0.7
lambda_edge, lambda_cx	1.0, 0.5
Local piece losses	piece GAN=1.0, feature matching=10.0, VGG=5.0

At inference time, the model still satisfies the Project 3 contract: only the FEN and viewpoint are needed. The system renders the synthetic board from the FEN, derives the same control maps from that synthetic state, forwards the model, and saves the required PNG outputs.

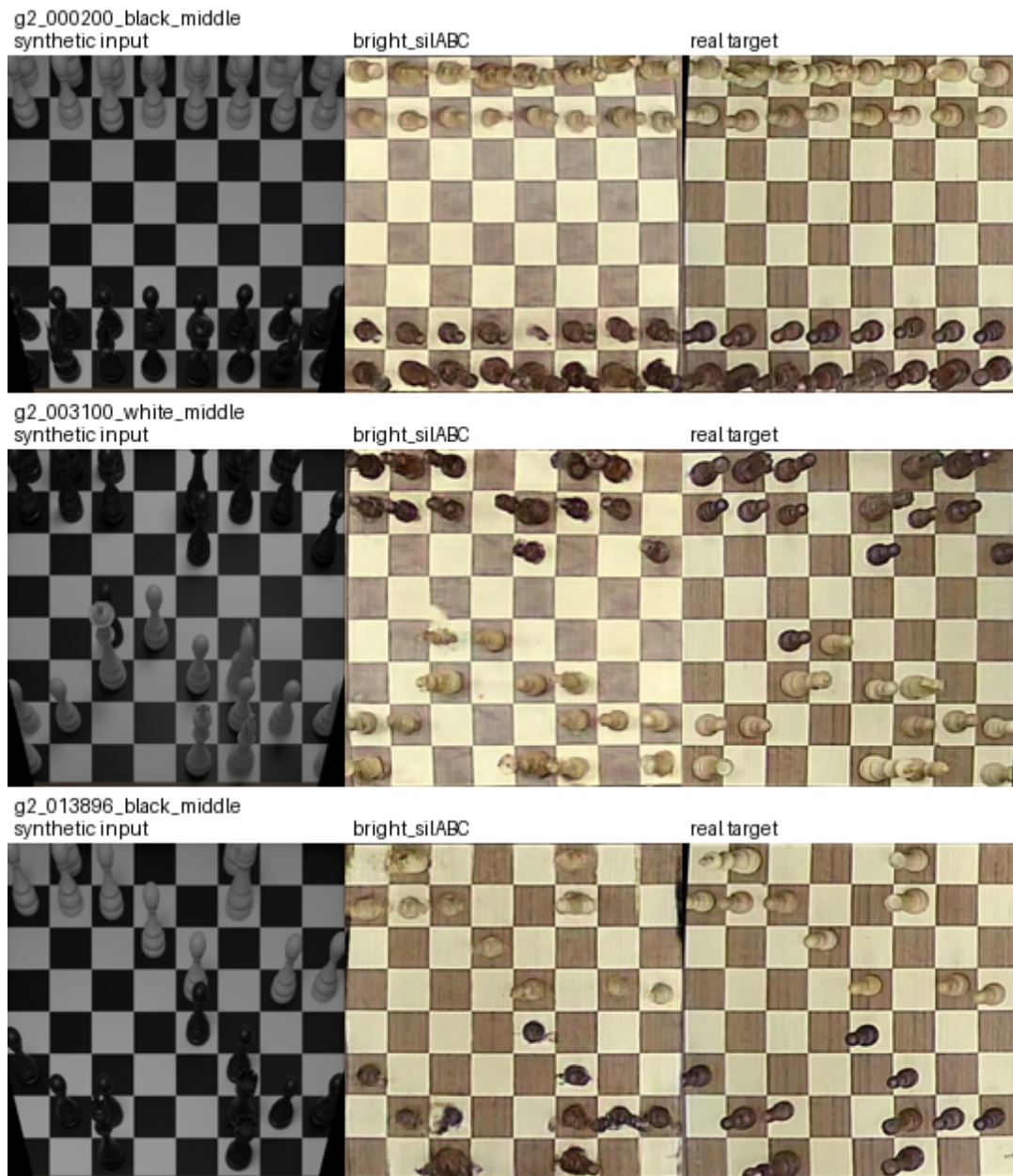


Figure 5. Final qualitative examples. Each row shows the Blender synthetic input, the selected bright_silABC output, and the real target image from the held-out test game.

5. Experiments

Dataset and splits

The training and test data, and the game-based train/test split, are described in Section 3 (Data and Preprocessing): 736 training pairs from games 4, 5, 6, 7 and 140 held-out test pairs from game 2. All results below are computed on those same 140 held-out game-2 boards.

Evaluation metrics: automatic and visual

We check the model's outputs in two different ways, because neither one alone is trustworthy.

The first is automatic: a computer program scores every generated board with no human looking at it. We use an independent classifier (a separate, smaller neural network trained only to recognize pieces in real photos, not the same network that generated the image) to read each square of every generated board and report the numbers defined in the legend below. This is fast, objective, and covers all 140 test boards.

Metric	Plain meaning
Square accuracy	Out of all 64 squares on a board, what fraction did the classifier get exactly right (correct piece type and colour, or correctly empty)?

Metric	Plain meaning
Occupancy accuracy	Out of all 64 squares, what fraction did it correctly call "has some piece" vs. "is empty", even if the exact piece type is wrong?
Type accuracy	Looking only at squares that truly have a piece, what fraction got the exact correct piece type (for example, specifically a black knight, not just "some piece")?
Whole-board exactness	occupancy Out of all 140 boards, what fraction had every single square's occupied-or-empty status correct, all at once? (A much stricter, all-or-nothing score per board.)
Phantom rate	Out of all squares that are truly empty, what fraction did the model incorrectly paint a piece onto?
Missing rate	Out of all squares that truly have a piece, what fraction did the model incorrectly leave looking empty?

The second is visual: a person looks at the actual generated images and checks for problems a score alone cannot catch. This matters because the automatic score can be misleading: if a pawn blurs together with the square behind it into one brown smear, the classifier may still guess "pawn" correctly from the blob's rough shape and position, even though a human looking at the same image immediately sees that it looks broken. So we also inspect grids of generated images by eye and run specialized audits that count specific visual defects: double heads (a piece that appears to have two heads or a ghost outline), halos (a faint glow around a piece that should not be there), transparency (a piece that looks see-through), and other piece-detail problems.

Baselines and comparisons

Our first baseline was unpaired translation (CUT and CycleGAN-style training, Sections 2 and 7); it was rejected before we built the automatic classifier, because its failure was visible directly in the generated images: pieces moved to different squares and texture appeared on empty squares. This is a structural failure of the unpaired approach itself, not a borderline case the classifier was needed to catch, so we did not score it with the classifier and report no number for it here; its rejection is documented qualitatively in Section 7. All numeric comparisons below are therefore between variants of our own paired model, built by changing one component at a time, which is the comparison that matters for justifying the final design (the required ablation study, Section 6).

What the run names mean

The names in Table 5 (for example `bright_silABC`) are not standard terms; they are our own short labels for which components of the model are switched on, listed in the legend below.

Label	Meaning
<code>sil</code>	Uses the silhouette-shaped semantic conditioning described in Section 4 (" <code>fen_silhouette</code> ").
<code>A / B / C</code>	Which of the three loss components from Section 4 (Training losses) are active: A = silhouette semantics, B = edge loss + masked L1, C = Contextual Loss. <code>silABC</code> has all three; <code>silAB</code> has A and B only (C, the Contextual Loss, is off).
<code>bright</code>	Uses the data fix that lowers the silhouette brightness threshold so the dark, anti-aliased edges of black pieces are captured (Section 4).
<code>noPieceD</code>	The local piece-crop discriminator (Section 4) is switched off; everything else is unchanged.
<code>pcomp_srcshape</code>	An experiment that generates the piece and the board background separately and blends them, with an added shape loss based on the synthetic source silhouette.
<code>geometry lock</code>	An experiment that forces the output to exactly match the rendered synthetic geometry.
<code>parallax8</code>	An experiment that stretches the synthetic piece silhouettes upward before fine-tuning, to compensate for parallax.

Quantitative results

Run	Square	Occupancy	Type	Whole-board occupancy	Phantom	Missing
<code>bright_silAB</code>	0.9160	0.9924	0.7849	0.6429	0.0057	0.0109
<code>bright_silABC (selected)</code>	0.9221	0.9916	0.8040	0.7571	0.0057	0.0130
<code>silAB noPieceD</code>	0.9314	0.9942	0.8240	0.7643	0.0052	0.0068
<code>pcomp_srcshape</code>	0.9235	0.9955	0.7979	0.7929	0.0019	0.0090

Run	Square	Occupancy	Type	Whole-board occupancy	Phantom	Missing
geometry lock train	0.6993	0.8738	0.3075	0.0000	0.0293	0.2991
parallax8 fine-tune	0.9094	0.9846	0.7879	0.4500	0.0167	0.0130

Table 5. Results on 140 held-out game-2 boards, from the independent classifier described above. Higher is better in every column except Phantom and Missing.

The selected model, `bright_silABC`, is not the winner of every scalar metric. For example, `silAB noPieceD` has slightly higher classifier scores. However, final selection used a stricter practical criterion: the model must preserve a realistic appearance and not introduce severe synthetic, milky, or transparent piece artifacts, which the classifier's numbers do not directly measure. `Bright_silABC` was selected as the best overall trade-off once both the numbers above and the visual audits (Section 6) were taken into account.

6. Ablation Study

The final method contains several components, so we performed ablations and negative tests to justify the design. The most important conclusion is that improving board-state metrics alone is not enough. Components that force exact geometry can remove double heads, but often destroy the photo-like style. Components that preserve style often leave the topology problem unsolved.

Ablation	Hypothesis	Observed result	Decision
Silhouette-only semantics	Visible silhouettes should encode piece layout	High phantom rate in the V5 probe; no whole-board occupancy exactness	Reject; full-cell FEN is necessary
Full-cell FEN + V5 geometry	FEN layout protects occupancy while geometry provides shape	Probe reached 1.0 occupancy on 16 boards	Use as V5 baseline
No piece classifier	Classifier loss might overfit or reward blobs	Improved some classifier metrics but did not clearly solve visual defects	Not final
Geometry lock	Hard source geometry will prevent double heads	Double-head defects dropped strongly, but pieces became washed and synthetic	Reject for final style
Two-stream / piece composite	Separate foreground and background generation	Style remained reasonable, topology mostly unchanged	Not enough
Parallax8 source stretch	Synthetic pieces are still too short or top-down	Worse occupancy, worse phantom rate, no double-head improvement	Reject

Run	Double-head score	Any-extra rate	Pawns any-extra	Officers any-extra
<code>bright_silAB</code>	2.4544	0.5342	0.5932	0.4481
<code>bright_silABC</code>	2.4004	0.5463	0.6236	0.4336
<code>parallax8</code>	2.5367	0.5593	0.6346	0.4496

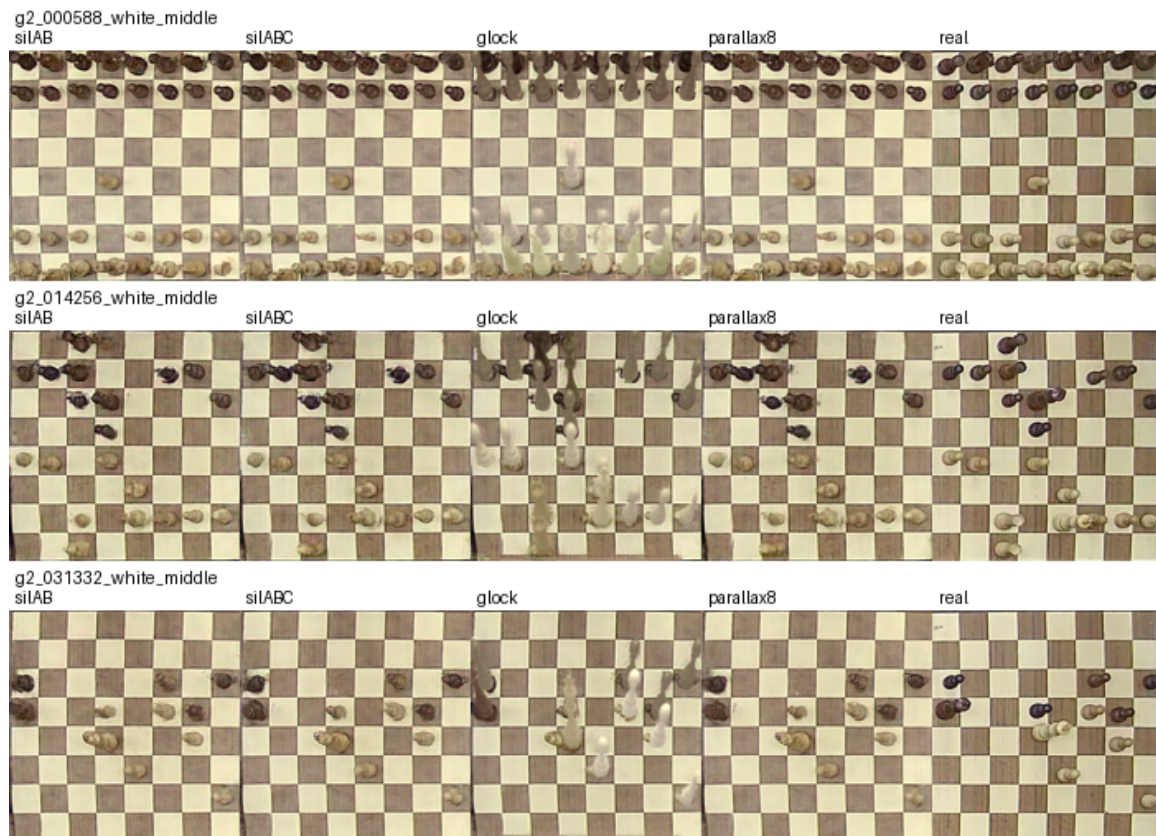


Figure 6. Qualitative ablation comparison. Geometry-lock improves structure but looks synthetic; parallax8 does not improve the selected bright_silABC model.

7. What Did Not Work

We document four attempts that did not make it into the final model, because honest negative results are part of good research practice and they explain design choices that would otherwise look arbitrary. Each one is broken down the same way: what we tried, why it did not work, what we thought going in might fix the problem, and what we did instead once it failed.

Attempt 1: Unpaired translation (CUT / CycleGAN)

What we tried	Trained an unpaired image-to-image model (CUT/CycleGAN-style) to translate the synthetic renders into the real photographic style, with no pixel-level pairing between a specific synthetic image and a specific real target.
Why it did not work	The model transferred wood-grain and lighting style convincingly, but it received no direct penalty for moving a piece to a different square or inventing piece-like texture on an empty square. The position itself was not reliably preserved, which is unacceptable for a task where the exact board state must remain correct.
What we thought might fix it	We considered whether stronger unpaired constraints (for example tighter cycle-consistency or extra identity losses) could be added on top of CUT/CycleGAN to force position preservation.
What we did instead	We concluded the limitation was structural, not a tunable hyperparameter: unpaired training has no notion of "this exact output pixel must match this exact target pixel". We pivoted to paired training (Sections 2 and 4), where every synthetic-real pair is matched and a direct loss can penalise this kind of drift.

Attempt 2: Naive parallax correction (parallax8)

What we tried	Starting from the already-trained bright_silABC checkpoint, we stretched the synthetic piece silhouettes upward before fine-tuning.
Why it did not work	It did not reduce the double-head audit score or the visual artifacts, and it degraded whole-board occupancy exactness from 0.757 to 0.450 (Table 5) – it made the model worse at something it was already doing well.

What we thought might fix it	We hypothesised that the synthetic pieces still looked too short or too top-down compared with the real, more oblique photographs, and that a taller synthetic silhouette would better match the real piece's true vertical extent after rectification, closing the geometry gap.
What we did instead	We reverted to the unmodified, already camera-aligned "bright" dataset (Section 3) and concluded that the remaining mismatch is a genuine 3D parallax effect that a 2D pixel-level stretch cannot correct. Closing it properly would need true camera/lens calibration or 3D-aware rendering, which we note as future work (Section 8), not a post-render image distortion.

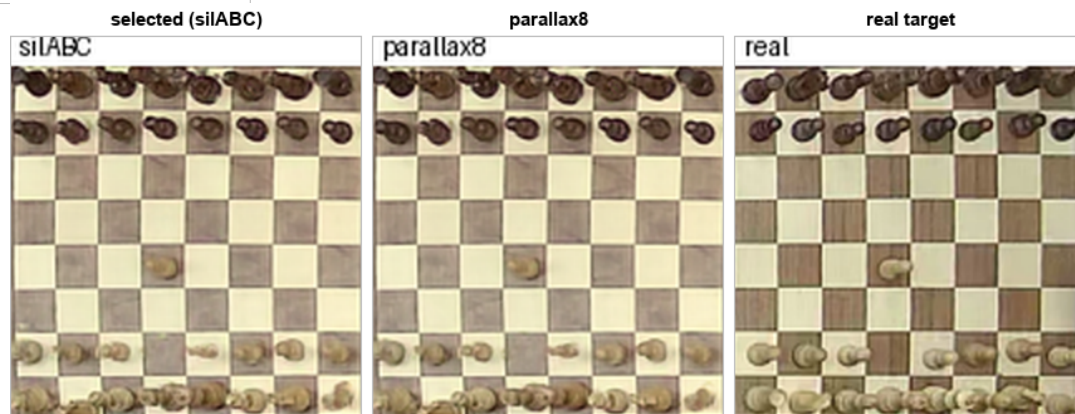


Figure 7. Parallax8 example (crop of Figure 6). The parallax8 column looks similar to the selected model rather than clearly better, while whole-board occupancy exactness measurably dropped (Table 5) – the stretch did not deliver the improvement it was meant to.

Attempt 3: Hard geometry locking

What we tried	Constrained the generator's output to follow the rendered synthetic geometry very closely (a hard geometry lock).
Why it did not work	Double-head defects did drop sharply, but only because the generator was forced toward the synthetic shape. The resulting pieces looked washed-out, transparent, or milky rather than photographic, and whole-board occupancy exactness collapsed to 0.000 (Table 5), failing the realistic-image requirement outright.
What we thought might fix it	We reasoned that double-head and ghosting artifacts come from the generator having too much freedom to invent structure, so removing that freedom by locking it to the known-correct synthetic shape should solve the topology problem without touching style.
What we did instead	We learned that clean topology and photographic style pull in opposite directions under our current losses: forcing one directly damages the other. We kept the geometry channels as a soft conditioning signal rather than a hard constraint, and documented the residual topology imperfections as a limitation (Section 8) instead of trying to force them away.

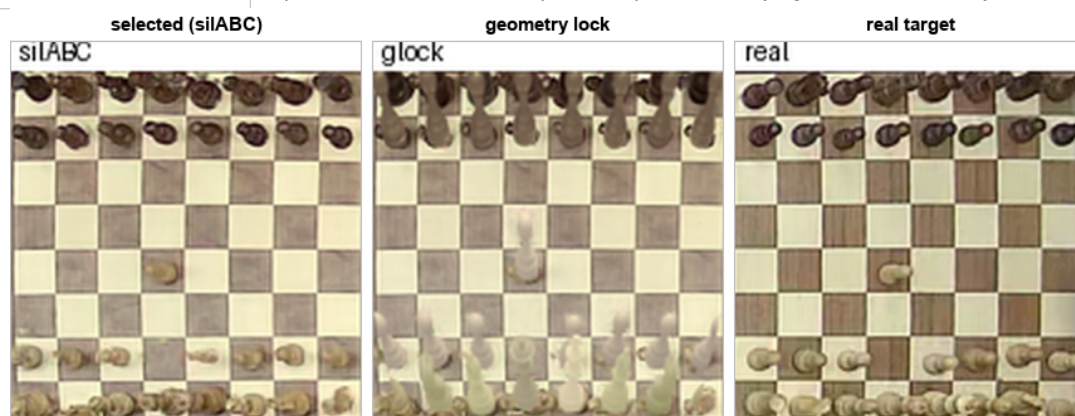


Figure 8. Geometry-lock example (crop of Figure 6). The geometry lock column shows a clear washed-out, milky cloud over several squares where the real target simply has wooden pieces – the visual symptom of forcing the output to match the synthetic shape too rigidly.

Attempt 4: Automatic real-piece masks (SAM + colour trimming)

What we tried	Used Segment Anything (SAM) together with colour-distance trimming to automatically extract clean piece masks directly from the real photographs, intended as a more precise geometry signal than the synthetic render alone.
----------------------	---

Why it did not work	At the available 40-60 pixel piece scale, many white-piece pixels are nearly the same colour as the light board squares. The automatic segmentation often captured the surrounding square along with the piece, or fragmented a single piece into several disconnected regions.
What we thought might fix it	We considered tuning SAM's prompts and the colour-trimming threshold to better separate piece from board at this scale.
What we did instead	We abandoned automatic real-photo masking and relied entirely on the synthetic render's silhouette (Section 4), which is exact by construction because it comes from the controlled Blender scene rather than from noisy real pixels.

8. Discussion and Limitations

The selected model is a practical submission model, not a complete solution to realistic chess piece synthesis. It generally preserves occupancy well and creates a visually plausible wooden board, but local piece geometry remains imperfect. Common failures include blob-like pawns, side-lobed heads, weak separation between neighboring pieces, and occasional phantom texture on empty squares.

The main limitation is the supervision itself. The real targets are board-rectified photographs of a 3D scene. A homography aligns the board plane, but not the elevated heads of pieces. Pixel and perceptual losses therefore supervise slightly incompatible shapes. Reducing L1 pressure on piece regions avoids severe blur, but also leaves the generator with weaker direct supervision for fine piece structure.

The most credible future improvement would be new supervision rather than more loss tuning: manual annotation of real piece masks, higher-resolution capture, or a renderer/camera calibration pipeline that matches the real camera pose and lens more precisely. With such data, one could train a dedicated foreground matting or piece-refinement model instead of asking a small GAN to infer ambiguous local topology from low-resolution photos.

9. Conclusion

We built a controlled synthetic-to-real chessboard generation pipeline for Project 3. The final model is a paired pix2pixHD-style generator with FEN, silhouette, depth, and edge conditioning. The selected bright_silABC checkpoint is trained on the camera-aligned bright V5 dataset and gives the best overall trade-off between realistic appearance and state preservation among the tested models. The project also produced a clear negative result: under the current low-resolution data and automatic supervision, there is a persistent trade-off between clean piece topology and realistic photographic style. This limitation is documented through both quantitative ablations and visual comparisons.

References

- [1] P. Isola, J.-Y. Zhu, T. Zhou, and A. A. Efros. Image-to-Image Translation with Conditional Adversarial Networks. CVPR, 2017.
- [2] T.-C. Wang et al. High-Resolution Image Synthesis and Semantic Manipulation with Conditional GANs. CVPR, 2018.
- [3] J.-Y. Zhu, T. Park, P. Isola, and A. A. Efros. Unpaired Image-to-Image Translation using Cycle-Consistent Adversarial Networks. ICCV, 2017.
- [4] T. Park, A. A. Efros, R. Zhang, and J.-Y. Zhu. Contrastive Learning for Unpaired Image-to-Image Translation. ECCV, 2020.
- [5] N. Ravi et al. Segment Anything. ICCV, 2023.
- [6] Blender Foundation. Blender, open-source 3D creation suite. <https://www.blender.org/>
- [7] Project repository: https://github.com/TalPodgor/DL_Project3